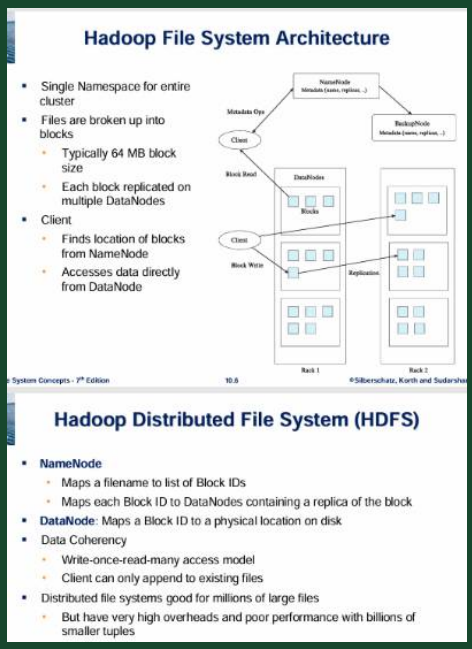
HDFS Architecture Explained



explain this in details

The image provides an overview of the **Hadoop File System Architecture (HDFS)** and its components. Let's go through the details step by step.

Hadoop File System Architecture

HDFS is a **distributed file system** designed for handling large volumes of data across multiple machines. It is fault-tolerant, scalable, and optimized for big data applications.

Key Features of HDFS Architecture

1. **Single Namespace for the entire cluster**
   - The entire Hadoop storage system appears as a **single logical file system** for users.
   - This simplifies data access and management.
2. **Files are broken into blocks**
   - Large files are **split into fixed-size blocks** (typically 64MB or 128MB, depending on configuration).
   - Each block is stored **across multiple DataNodes** to ensure redundancy and fault tolerance.
3. **Client Interactions**
   - The **Client** communicates with the **NameNode** to get metadata (block locations).
   - After retrieving metadata, the Client **directly accesses the DataNode** to read/write data.

Hadoop File System Components

The diagram illustrates three main components:

1. **NameNode (Metadata Manager)**
   - Stores metadata (filenames, block locations, replication information).
   - Maps filenames to lists of **Block IDs**.
   - It does not store actual file data—only metadata.
   - If the NameNode fails, the system can become unavailable unless a **BackupNode** or **Secondary NameNode** is in place.
2. **DataNodes (Storage Workers)**
   - Store actual data blocks on disk.
   - Each block is replicated across multiple DataNodes.
   - NameNode instructs DataNodes on where to store and replicate data.
   - Clients read/write data directly from DataNodes.
3. **BackupNode (Secondary NameNode)**
   - Keeps a copy of metadata from the NameNode.
   - Helps in failure recovery but does not replace the NameNode entirely.

Hadoop Distributed File System (HDFS)

HDFS is the core storage system of Hadoop, designed to handle large-scale distributed data storage.

Components and Their Roles

1. **NameNode**
   - Centralized metadata manager.
   - Maps a filename to its respective **Block IDs**.
   - Tracks the location of each block replica across DataNodes.
2. **DataNodes**
   - Store actual data blocks.
   - Map **Block IDs to physical locations** on disk.
   - Perform replication as instructed by the NameNode.

Data Coherency Model

- HDFS follows a **write-once, read-many** model.
- Clients **cannot modify existing files** but can **append new data** to files.

Scalability and Performance Considerations

- **Optimized for large files** (millions of files can be stored efficiently).
- **Not suitable for small files** (overhead is high for managing a large number of small-sized tuples).

How Data Flows in HDFS

1. **Writing Data**
   - The client contacts the NameNode for metadata.
   - The NameNode assigns block locations.
   - The client writes data directly to the assigned DataNodes.
   - Data is replicated across multiple nodes.
2. **Reading Data**
   - The client requests metadata from the NameNode.
   - The NameNode provides block locations.
   - The client fetches data directly from the DataNodes.

Summary

- HDFS provides **fault-tolerant** and **distributed storage** for big data applications.
- Data is split into **large blocks** and **replicated** across multiple DataNodes.
- The **NameNode manages metadata**, while **DataNodes store the actual data**.
- The system is **highly scalable** but **not ideal for small files** due to high overhead.
- HDFS follows a **write-once, read-many** policy with an **append-only** mechanism.

Would you like me to explain any part in more detail?